

POLY MORPHISM:

POLY MORPHISM
↓ ↓
ways ways
 to
 represent
 (forms)

So the act of representing same thing in multiple ways like a constructor overloading if we have 3 constructors one take one argument other take two to three arguments so it will decide at runtime which constructor is called. This is an example of poly morphism.

Types of polymorphism:

① Compile Time / Static Polymorphism:

achieve by method overloading



same name of method
but type of return, no of arguments can be changed.

Why called compile time. Java actually determines at the compile time which method would be called.

~~like at compile time it will see which constructor should be called, same as if method is overridden in 2, 3 different classes it should be seen which class method should be overloaded. ~~overridden~~ method should be called.~~

~~method overloading is done in one class only. but can be in diff~~

~~Runtime / Dynamic Polymorphism:~~

→ Method overloading is done in on class only but can be in different inherited class if the rules of overloading applied otherwise it would be overriding.

② Runtime or Dynamic Polymorphism

It decides on runtime which class method would be called like parent or child class.

like if
public circle extends shape {

@Override
void area() { } }

Consider same name method with same return type and same no of arguments in shape class then ... it decide on runtime with reference that which area method would be triggered.

→ what method is being called depends on the reference type not on basis of object.

means if type is parent class reference then it will be of parent method

Parent P = new Child();
↓
This method is called
(area)

→ final keyword prevents overriding.

→ private keyword also prevent overriding.

Even if the class is ~~inherited~~
a final declared class it
could not be inherited.

Also if we make a
class final all of its
methods would be final
as well.

Q Can static methods be overridden?

No, Static methods could not
be overridden ~~because it made~~
~~a new method in other~~
because the method in parent
class will always run from the
class because it does not
depend on the object.

They can be inherited but cannot
be overridden.